

GÉNÉRATION ALÉATOIRE DE MOT DE PASSE 1

Programmation C++ Linux



Lycée Diderot, Paris

BTS CIEL 1

TABLE DES MATIÈRES

Table des illustrations	2
Introduction.....	3
Format d'un fichier source en C++	3
Les étapes de la compilation	4
Fiche de tests.....	6
Identifier le besoin	6
Focaliser sur les parcours stratégiques	6
Imaginer les comportements consommateurs les plus insolites.....	6
Séquencer et identifier les actions.....	7
Etre précis sur les attendus	7
Génération de mot de passe	8
Diagrammes UML.....	8
Cas d'utilisation	8
Séquence	8
Prérequis	9
Création et utilisation de variables	9
Types de variables	9
Dialoguer avec l'utilisateur	10
Parcours d'une chaîne de caractères	10
Travail à réaliser	10
Github.....	10
Index.....	12

TABLE DES ILLUSTRATIONS

Figure 1: Structure d'un programme en C++	3
Figure 2: Diagramme des cas d'utilisation	8
Figure 3: Diagramme de séquence.....	9

INTRODUCTION

Le développement logiciel et ses bases vont vous être utiles quel que soit le domaine informatique vers lequel vous vous spécialiserez, que ce soit pour faire des petits scripts pour se faciliter la tâche ou des programmes complexes pour les géants des GAFAM.

Le langage que j'ai choisi pour commencer est le **C++**. Vous serez amenés à en voir d'autres dans le futur. Le C++ est, avec son aîné le C, un langage toujours très utilisé dans le monde. (cf [le classement TIOBE](#))

Il existe pléthore de langages informatiques, chacun présentant ses avantages et ses inconvénients. Celui-ci n'est ni pire ni meilleur. Le but du jeu est d'acquérir, tout au long des TP proposés, une méthode de pensée et de conception. Passer sur un autre langage ne posera alors que des problèmes de syntaxes, pas de philosophie.

FORMAT D'UN FICHIER SOURCE EN C++

Un code source peut être constitué de plusieurs **fichiers source** (.cpp) et également de plusieurs **fichiers d'en-tête** (.h). Il doit avoir obligatoirement la fonction principale **main** pour pouvoir créer un exécutable. Je vous donne un code source simple, le fameux **hello world** que tout débutant en informatique se doit de taper.

```
#include <iostream>
using namespace std;
int main ()
{
    cout<< "Bonjour à vous!" <<endl;
    return 0;
}
```

Figure 1: Structure d'un programme en C++

Ce code est constitué de plusieurs instructions et directives de compilation. Les instructions sont repérées par le point-virgule qui les termine. Les directives de compilation sont repérées par le dièse qui les précède.

Ici, on commence par une directive, on dit au compilateur que l'on utilise la librairie **iostream** dans notre programme. Elle comprend la définition de l'opérateur **cout** qui permet d'afficher un message à l'écran.

On retrouve également la fonction **main** indispensable à tout code source.

On sauve ce code dans un fichier **bonjour.cpp**, qui nous permettra, une fois compilé, d'avoir un fichier exécutable affichant "Bonjour à vous!".

Comme dit plus haut, le **C++** est un langage compilé ce qui veut dire que l'on a besoin d'un compilateur pour transformer notre code source en suite d'instructions simples pour le processeur.

LES ÉTAPES DE LA COMPILATION

Un langage informatique est un langage précis, avec sa grammaire et ses règles. On observe deux types principaux de langages: le compilé et l'interprété. Un langage dit compilé est un langage dont le code doit subir une transformation et ainsi obtenir un exécutable. A contrario, un langage interprété sera exécuté au fur et à mesure de la lecture du code source.

Le **C++** est un langage compilé. Il faut donc un compilateur c'est à dire un programme capable de traduire le langage informatique (autrement dit humain) en langage machine.

g++ est le compilateur de base du **C++** (pour le **C**, le compilateur se nomme **gcc**). Il évolue en permanence. Certaines choses impossibles il y a seulement 6 mois peuvent l'être aujourd'hui. Cela ne veut pas dire que l'on codait mal 6 mois plus tôt, juste que le compilateur est modifié en vue de simplifier la vie du développeur.

Nous allons donc utiliser **g++** pour compiler nos futurs codes source. Sa syntaxe est simple, nous utiliserons celle qui suit:

```
g++ [fichiers source à compiler] -o [nom de l'exécutable]
```

Sous **Linux**, un exécutable ne porte pas forcément l'extension **.exe** propre à **Microsoft**. Dans la syntaxe ci-dessus, on utilise une option de compilation, le **-o**. Elle est utilisée pour permettre de donner un nom à l'exécutable généré. Si vous ne l'utilisez pas, vous aurez un exécutable répondant au doux nom de **a.out**. Il existe d'autres options, si vous êtes curieux, tapez à l'invite de commande **man g++** et vous obtiendrez la page de manuel concernant le compilateur avec toutes ses options.

Une option également utile pour nous est le **-c**. On oblige notre compilateur à seulement compiler notre code source. Comment ça seulement compiler, me direz-vous? Depuis le début, je dis que la compilation génère un exécutable. En fait, pas tout à fait. La compilation telle qu'on la conçoit se déroule en deux étapes: la compilation et l'édition de lien.

La compilation est en fait l'étape qui va vérifier que votre code source est bien écrit et qui va le traduire dans le langage machine. Si vous n'exécutez que cette étape, vous obtiendrez de votre compilation un fichier objet, autrement dit, un fichier binaire traduisant l'ensemble de vos instructions en **C++** et instructions simples pour le processeur.

Pour obtenir un exécutable, il faut éditer les liens. C'est à dire que votre code source se base sur de l'existant, vous n'inventez pas (pour l'instant), vous utilisez des fonctionnalités déjà

connues. Il faut donc que le compilateur intègre ces fonctionnalités issues de bibliothèques dans votre code afin d'obtenir un exécutable.

Qui dit deux étapes dit deux origines d'erreur, celle de compilation et celle d'édition de lien. Les erreurs de compilation concerneront uniquement les erreurs de syntaxe de votre code source (ce qui ne veut pas dire que c'est simple à déboguer). Les erreurs d'édition de liens concerneront l'intégration des liens, à savoir par exemple, une fonction manquante ou carrément un fichier objet manquant (là non plus, ce n'est pas simple à déboguer).

Dans notre exemple plus haut, la ligne de commande deviendrait: **g++ bonjour.cpp -o bonjour**

Pour lancer l'exécutable, il suffit de taper à l'invite de commande **./bonjour**

Vous pouvez également générer le code assembleur à partir de votre fichier source. L'assembleur est le langage de plus bas niveau qui soit, comprendre le langage le plus proche du fonctionnement de la machine mais lisible par un humain. Il n'y a pas un assembleur mais des assembleurs, ce langage dépendant fortement de l'architecture matérielle et logicielle. Pour voir à quoi cela ressemble, il suffit de taper à l'invite de commande **g++ bonjour.cpp -S**

Cela générera un fichier **bonjour.s** au nombre de lignes plus conséquent.

FICHE DE TESTS

Une fiche de test est un outil particulièrement utile dès lors que l'on développe et/ou installe un outil informatique.

Quelques étapes sont nécessaires pour établir la fiche de test optimale. En exemple, reprenons le fameux Hello World!

IDENTIFIER LE BESOIN

Vous devez comprendre et savoir ce que doit réaliser votre outil: cela permet de ne pas faire du hors sujet. Ici, on veut un programme d'accueil qui soit poli. A terme, on veut aussi pouvoir s'identifier.

FOCALISER SUR LES PARCOURS STRATÉGIQUES

Vous devez vous demander quels sont les différentes utilisation possibles (ou scénarii) de votre application. Cela vous permettra de focaliser votre attention sur les tests importants à réaliser pour s'assurer du fonctionnement correct de votre application. Ici, le scénario est simple: l'application nous accueille et nous demande notre nom.

IMAGINER LES COMPORTEMENTS CONSOMMATEURS LES PLUS INSOLITES

Le problème en informatique est très souvent situé entre la chaise et le clavier. Il vous faut donc essayer de penser aux utilisations farfelues de votre programme qui peuvent se produire. Vous pouvez être très imaginatifs, vous n'arriverez jamais à penser à tout. Comme penser à écrire dans une notice d'utilisation de perche à selfie que son usage n'est pas recommandé par temps orageux en présence de tonnerre et d'éclairs.

SÉQUENCER ET IDENTIFIER LES ACTIONS

Une fiche de test bien rédigée doit permettre à des gens extérieurs au projet, voire à des utilisateurs, de pouvoir réaliser tous les tests facilement.

Nom de la fiche : Hello World

Objectif : Vérifier le bon fonctionnement de l'affichage des messages et la saisie des informations

Prérequis : Aucun

N°	Action	Attendu	Résultat
1	Dans un terminal, lancer la commande <code>g++ bonjour.cpp -o bonjour</code>	Aucun message ne s'affiche	OK
2	Dans un terminal, lancer la commande <code>ls</code>	Les fichiers <code>bonjour.cpp</code> et <code>bonjour</code> sont listés et affichés	OK
3	Dans un terminal, lancer la commande <code>./bonjour</code>	Un message d'accueil s'affiche: Bonjour à vous!	OK
4	Dans un terminal, lancer la commande <code>g++ bienvenue.cpp -o bienvenue</code>	Aucun message ne s'affiche	OK
5	Dans un terminal, lancer la commande <code>./ bienvenue</code>	Un message d'accueil s'affiche: Bonjour, veuillez saisir votre nom!	
6	Saisir son nom (eg: Tom) à la demande du programme	Un message répétant le nom saisi est affiché: Bienvenue Tom!	OK

ETRE PRÉCIS SUR LES ATTENDUS

Le plus dur est de bien penser aux attendus des tests que vous réalisez: par exemple, que se passe t'il si on saisit un nombre au lieu d'un nom? L'attendu ici se doit être très précis sans aucune place à l'interprétation: soit on fait en sorte de détecter ce cas de figure, soit on considère que la personne peut avoir un nom étrange, comme le fils d'un milliardaire célèbre.

Il faut penser au fonctionnement normal de l'application mais également au fonctionnement anormal ou à l'utilisation anormale de l'application. Plus tôt vous y pensez, plus tôt vous l'intégrez dans votre code, plus robuste votre application est.

GÉNÉRATION DE MOT DE PASSE

Notre objectif, pour les prochains TP, va être de créer dans un premier temps un générateur de mot de passe, un peu comme sur vos navigateurs et ensuite de recréer un programme existant (crunch) permettant de générer tous les mots de passe possibles à partir d'un alphabet spécifié et d'une fourchette pour le nombre de caractères.

DIAGRAMMES UML

CAS D'UTILISATION

On va s'intéresser au diagramme **UML** (Unified Modeling Language) des cas d'utilisation de notre application.

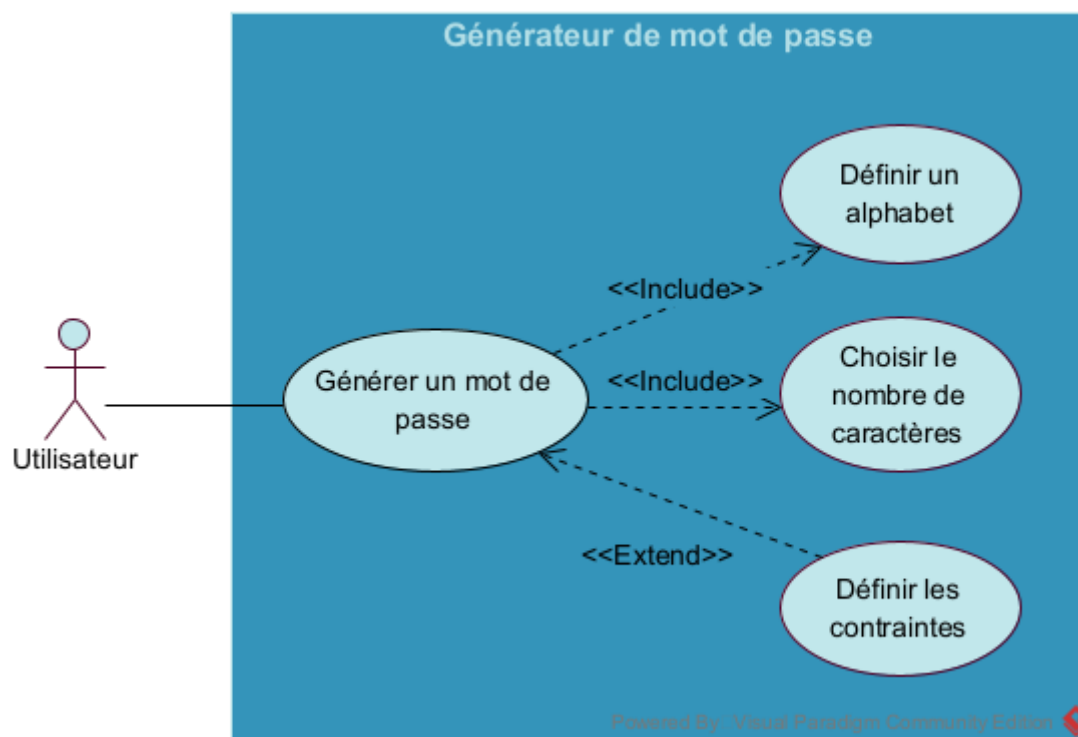


Figure 2: Diagramme des cas d'utilisation

Ce diagramme nous montre ce à quoi on va aboutir au bout de plusieurs TP même s'il est suffisamment simple pour finalement couvrir ce qui est le but de ce premier TP.

La génération d'un mot de passe sous-entend que cela se fait de manière aléatoire: **pas pour aujourd'hui**, cela fera l'objet d'un TP prochain.

SÉQUENCE

Un diagramme de séquence a pour objectif de montrer l'interaction entre les différents objets d'un système. Ici, cela reste relativement simple.

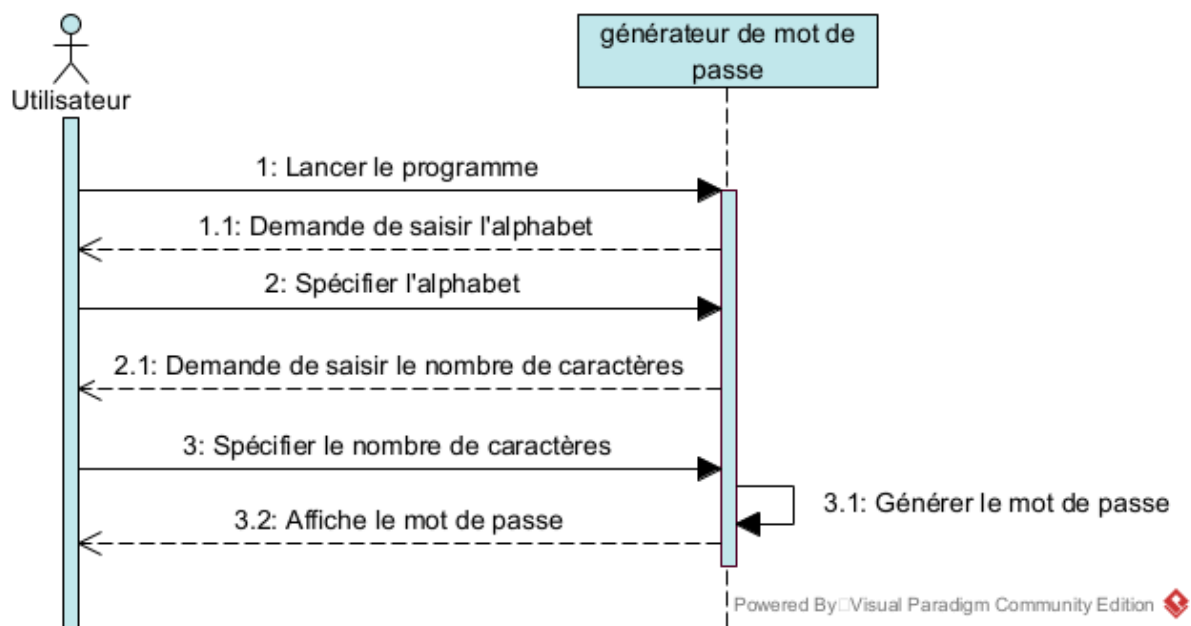


Figure 3: Diagramme de séquence

PRÉREQUIS

CRÉATION ET UTILISATION DE VARIABLES

L'alphabet à utiliser, le nombre de caractères pour le mot de passe, le mot de passe doivent forcément être stockés afin de pouvoir être traités plus facilement.

Il faut donc stocker toutes ces informations dans des variables.

Elles permettent de réserver un emplacement mémoire dans la **RAM** (Random Access Memory) afin de pouvoir y stocker tout type d'information.

TYPES DE VARIABLES

Le **C++** est un langage fortement typé: comprenez que contrairement à des langages comme **Python** ou **PHP**, vous devez spécifier le type de données que votre variable va contenir.

Dans notre cas et pour l'instant, nous souhaitons stocker des caractères pour l'alphabet et le mot de passe et un nombre pour le nombre de caractères à utiliser. On a affaire ici à des chaînes de caractères et à un nombre entier.

Il est à noter que quel que soit le type de variable que l'on doit utiliser dans un programme, sa déclaration se fait toujours ainsi: **type nomDeLaVariable;**

On utilisera le type **string** pour les chaînes et **unsigned short** pour le nombre de caractères.

DIALOGUER AVEC L'UTILISATEUR

Si **cout** est l'opérateur de sortie à l'écran, il en faut bien un pour l'entrée au clavier. Il s'agit de **cin**. Son utilisation est simple, il suffit d'écrire l'instruction ainsi: **cin>>maVariable;**

PARCOURS D'UNE CHAÎNE DE CARACTÈRES

Une chaîne de caractères peut aussi être appelée tableau de caractères. De fait, on peut très bien afficher une chaîne caractère par caractère.

Par exemple, pour afficher tous les caractères d'une chaîne, je peux écrire:

```
cout<< chaine[0];  
cout<< chaine[1];  
...  
cout<< chaine[14];
```

Je peux accéder à ces valeurs grâce à une structure itérative, le **for**. Il s'agit d'une boucle qui se répète un nombre de fois fini. Sa syntaxe est :

```
for ([condition de départ];[condition de sortie];[pas])  
{  
    instructions;  
}
```

TRAVAIL À RÉALISER

La première chose à faire est la fiche de tests (ou de recette). Je vais pour ce premier TP vous en fournir une. Il n'est pas impossible que vous ayez des idées intéressantes (ça peut arriver, il ne faut pas désespérer, les miracles, ça arrive) et donc, vous pouvez les rajouter. Cette fiche vous donnera une idée claire de ce qu'il faut réaliser pour ce premier TP. La fiche est également fournie sur un document à part modifiable.

GITHUB

Github exécutera des tests sur vos codes à chaque push. Vous ne pouvez voir le résultat que sur le site et pas avec **Github Desktop**. Sur votre dépôt, vous avez un onglet **Actions**, c'est là que vous pourrez voir ce qui ne va pas.

Nom de la fiche : TP Génération de mot de passe 1

Objectif : Vérifier le bon fonctionnement de la génération non aléatoire d'un mot de passe

Prérequis : Aucun

N°	Action	Attendu	Résultat
1	Dans un terminal, lancer la commande <code>g++ genemdp.cpp -o genemdp</code>	Aucun message ne s'affiche	
2	Dans un terminal, lancer la commande <code>ls</code>	Les fichiers <code>genemdp.cpp</code> et <code>genemdp</code> sont listés et affichés	
3	Dans un terminal, lancer la commande <code>./ genemdp</code>	Un message d'accueil s'affiche et demande de saisir un alphabet	
4	Saisir l'alphabet à la demande du programme	Un message demandant de saisir le nombre de caractères du mot de passe est affiché	
5	Saisir le nombre de caractères du mot de passe	Le mot de passe est généré et affiché	
6	Refaire l'étape 3 et à l'étape 4 saisir un alphabet comportant des chiffres et des caractères spéciaux	Un message demandant de saisir le nombre de caractères du mot de passe est affiché	
7	Refaire les étapes 3 et 4 et à l'étape 5, saisir un nombre de caractères supérieur au nombres de caractères de l'alphabet saisi	Le mot de passe est généré et affiché (eg: alphabet: abc, nb de caractères: 5, mdp: abcab)	
8	Refaire les étapes 3 et 4 et à l'étape 5, saisir un nombre de caractères négatif	Rien ne s'affiche	



ATTENTION

A chaque séance, vous devrez faire une sauvegarde de votre travail sur Github avec Github Desktop par un commit puis un push. Ne pas le faire vous vaudra un 0. Ne pas rendre de travail dans le temps imparti vous vaudra 0. Ne pas avoir la coche verte signifie que le code rendu ne satisfait pas tout à fait aux attendus mais ne signifie pas pour autant une mauvaise note. L'objectif est d'avoir la coche verte.

Ne pas m'appeler pour valider les différents exercices vous vaudra un 0.

INDEX

C++, 3, 4, 9
cin, 10
cout, 3, 10
g++, 4, 5

RAM, 9
string, 10
UML, 8
unsigned short, 10